Managing Multicollinearity

Remove or Combine, PCA, Ridge Regression

Durjoy Kumar Sarker

Table of contents

1 Why This File? 2

2 Data 2
2.1 The problem (recap) . . . 3

3 Fix 1: Collect More Data 3

4 Fix 2: Remove One Variable 4

5 Fix 2b: Combine Variables into One 5

6 Fix 3: PCA (Principal Component Analysis) 6
6.1 Intuition first . . . 6
6.2 Step 1: Load the stats library . . . 6
6.3 Step 2: Standardize the variables . . . 6
6.4 Step 3: Run PCA . . . 7
6.5 Step 4: Use PC1 in regression . . . 7

7 Fix 4: Ridge Regression 9
7.1 Intuition first . . . 9
7.2 What is the penalty? . . . 10
7.3 Why does shrinking help? . . . 10
7.4 Lambda: the tuning key . . . 10
7.5 Code in R . . . 11
7.5.1 Read the data . . . 11
7.5.2 Prepare predictor matrix and response vector . . . 11
7.5.3 Fit ridge regression with cross-validation to find the best lambda . . . 12
7.5.4 Extract the best lambda value . . . 13
7.5.5 Predict yield using the ridge model and best lambda . . . 13

7.6 Seeing the shrinkage 13
7.7 How well does it predict? 14

8 Which Fix to Use? 15

9 Key Takeaways 15

1 Why This File?

File 02 showed how to **detect** multicollinearity (high correlation between predictors, high VIF). This file shows how to **manage** it.

If I have high multicollinearity, the options are:

1. **Collect more data**
2. **Remove one variable** (or combine variables into one)
3. **PCA** (Principal Component Analysis)
4. **Ridge regression**

```
library(car)
library(glmnet)
library(knitr)
```

2 Data

```
data <- read.csv("../data/raw/high_correlation_agriculture.csv")
head(data)
```

```
  yield nitrogen rainfall temperature
1 5.561    84.89   580.76       25.65
2 5.374    73.06   570.03       24.69
3 6.096    89.08   609.58       23.65
4 6.222    89.55   645.73       24.70
5 5.911    88.63   602.24       23.36
6 5.293    77.18   609.65       25.23
```

2.1 The problem (recap)

```r
model_full <- lm(yield ~ nitrogen + rainfall, data = data)
summary(model_full)
```

```
Call:
lm(formula = yield ~ nitrogen + rainfall, data = data)

Residuals:
     Min       1Q   Median       3Q      Max
-0.52047 -0.16536 -0.00482  0.17291  0.47635

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.134939   0.389605  -0.346   0.7306
nitrogen     0.044817   0.007522   5.958 3.11e-07 ***
rainfall     0.003272   0.001551   2.110   0.0402 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2211 on 47 degrees of freedom
Multiple R-squared:  0.9566,    Adjusted R-squared:  0.9548
F-statistic: 518.2 on 2 and 47 DF,  p-value: < 2.2e-16
```

```r
vif(model_full)
```

```
nitrogen rainfall
16.11817 16.11817
```

High VIF means nitrogen and rainfall carry **overlapping information**. Now we fix it.

3 Fix 1: Collect More Data

- More observations give the model more chances to separate the effect of one predictor from the other.
- It reduces the standard errors, but it does not remove the overlap itself.
- It is the best fix when it is possible, but in practice it is often not (cost, time, one season only).

So we move to fixes we can apply to the data we already have.

4 Fix 2: Remove One Variable

Intuition: if two predictors say the same thing, keep one.

```
model1 <- lm(yield ~ nitrogen, data = data)
summary(model1)
```

```
Call:
lm(formula = yield ~ nitrogen, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-0.5139 -0.1446  0.0015  0.1284  0.5511

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.60468    0.17600   3.436  0.00123 **
nitrogen     0.06019    0.00194  31.029  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2289 on 48 degrees of freedom
Multiple R-squared:  0.9525,    Adjusted R-squared:  0.9515
F-statistic: 962.8 on 1 and 48 DF,  p-value: < 2.2e-16
```

Compare with the two-variable model:

```
cmp <- data.frame(
  Model = c("nitrogen + rainfall", "nitrogen only"),
  R2 = c(summary(model_full)$r.squared, summary(model1)$r.squared),
  Adj_R2 = c(summary(model_full)$adj.r.squared, summary(model1)$adj.r.squared),
  AIC = c(AIC(model_full), AIC(model1))
)
kable(cbind(cmp[1], round(cmp[-1], 4)))
```

Model	R2	Adj_R2	AIC
nitrogen + rainfall	0.9566	0.9548	-4.1181
nitrogen only	0.9525	0.9515	-1.5937

Read it like this:

- If R^2 and adjusted R^2 barely change after removing a variable, that variable was **redundant**.
- Now the remaining predictor can show its real effect (smaller standard error).
- Downside: you lose the ability to say anything about the removed variable.

5 Fix 2b: Combine Variables into One

```r
data$combined <- (data$nitrogen + data$rainfall) / 2
model2 <- lm(yield ~ combined, data = data)
summary(model2)
```

```
Call:
lm(formula = yield ~ combined, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-0.5192 -0.2198 -0.0475  0.2429  0.4879

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.5529946  0.2838940   -5.47  1.6e-06 ***
combined     0.0204995  0.0007666   26.74  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2635 on 48 degrees of freedom
Multiple R-squared:  0.9371,    Adjusted R-squared:  0.9358
F-statistic:   715 on 1 and 48 DF,  p-value: < 2.2e-16
```

Intuition: instead of two overlapping variables, build **one** summary variable (here, their average).

Caution: nitrogen is in kg/ha and rainfall is in mm, so they are on different scales. A plain average lets the larger-valued variable dominate. This is exactly the problem that **standardizing** fixes in PCA, which is the next fix.

6 Fix 3: PCA (Principal Component Analysis)

6.1 Intuition first

- Nitrogen and rainfall move together, so they are really **one story told twice**.
- PCA builds **new variables (principal components)** that are **uncorrelated** with each other.
- **PC1** captures the biggest shared pattern, **PC2** the next biggest, and so on.
- Instead of using the original overlapping variables, we use PC1 (and more PCs only if needed).

6.2 Step 1: Load the stats library

```
library(stats)
```

It contains `prcomp()`, the function that performs PCA.

6.3 Step 2: Standardize the variables

```
vars <- scale(data[, c("nitrogen", "rainfall")])
head(vars)
```

```
         nitrogen     rainfall
[1,] -0.254912036 -0.785999792
[2,] -0.956640844 -0.917219884
[3,] -0.006370725 -0.433552163
[4,]  0.021508611  0.008536032
[5,] -0.033063706 -0.523315022
[6,] -0.712251774 -0.432696114
```

What `scale()` does:

- Subtracts the mean.
- Divides by the standard deviation.

Why standardize?

- PCA is affected by the scale of variables. For example, rainfall may be in the 100s and nitrogen in the 10s.

- Bigger values can dominate the PCA.
- Standardizing gives **equal weight** to all variables.

6.4 Step 3: Run PCA

```
pca_result <- prcomp(vars)
summary(pca_result)
```

```
Importance of components:
                          PC1     PC2
Standard deviation     1.4030 0.17753
Proportion of Variance 0.9842 0.01576
Cumulative Proportion  0.9842 1.00000
```

`prcomp()` returns:

- **PC1, PC2, ...**: new uncorrelated variables
- **How much variance each PC explains**

Check that the PCs are uncorrelated (this is the whole point):

```
round(cor(pca_result$x), 4)
```

```
    PC1 PC2
PC1   1   0
PC2   0   1
```

6.5 Step 4: Use PC1 in regression

```
modelpca <- lm(yield ~ pca_result$x[, 1], data = data)
summary(modelpca)
```

```
Call:
lm(formula = yield ~ pca_result$x[, 1], data = data)

Residuals:
     Min       1Q   Median       3Q      Max
```

```
-0.52255 -0.17486  0.01394  0.17720  0.41045

Coefficients:
                  Estimate Std. Error t value Pr(>|t|)
(Intercept)        5.97256    0.03215  185.75   <2e-16 ***
pca_result$x[, 1]  0.72343    0.02315   31.25   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2274 on 48 degrees of freedom
Multiple R-squared:  0.9531,    Adjusted R-squared:  0.9522
F-statistic: 976.5 on 1 and 48 DF,  p-value: < 2.2e-16
```

- `pca_result$x[, 1]` is the **score of PC1 for each row**.
- Instead of 2 (or 3) overlapping variables, you now use **1 new variable** that summarizes them.

Think of it like this: the original variables were a **noisy duplicate**, and PC1 is the **clean summary**.

What this gives you:

- Removes multicollinearity (PCs are uncorrelated).
- Improves model stability.
- Simplifies the analysis.

Trade-off: PC1 is a mix of variables, so it is harder to explain in agricultural terms ("one unit of PC1" is not "one kg of nitrogen"). Use the loadings to describe what PC1 represents.

eory example: three variables

The same steps with three variables (nitrogen, rainfall, temperature):

```r
vars3 <- scale(data[, c("nitrogen", "rainfall", "temperature")])
pca3  <- prcomp(vars3)
kable(round(summary(pca3)$importance, 4))
```

	PC1	PC2	PC3
Standard deviation	1.4078	0.9935	0.1764
Proportion of Variance	0.6606	0.3290	0.0104
Cumulative Proportion	0.6606	0.9896	1.0000

Example of how to read such a table (the numbers in your theory notes):

PC	Proportion of variance
PC1	0.85
PC2	0.12
PC3	0.03

- PC1 alone explains 85% of the variation, so PC1 is a very good summary of the original variables.
- Check the **cumulative proportion** row before deciding how many PCs to keep.## Th

Final tips:

- Aim to explain **80% or more** of the variance with as few PCs as possible.
- Start simple (PC1); add more PCs only if accuracy or model fit is poor.
- Always check the cumulative variance in the PCA summary before deciding.

7 Fix 4: Ridge Regression

7.1 Intuition first

- Ridge regression is an **improved version of linear regression**.
- Normal regression only tries to fit the data as closely as possible. With overlapping predictors, it can do this with **huge, unstable coefficients** (one big positive, one big negative, cancelling each other).
- Ridge adds a **penalty** (L2 penalty) that says: *"fit the data well, but keep the coefficients small."*
- The goal is to **shrink large coefficients**.

It helps when:

- Variables are correlated (multicollinearity).
- There are too many predictors (risk of overfitting).

Properties:

- Keeps **all** variables in the model (does not remove any).
- Controls and stabilizes coefficient values (a coefficient shows how strongly a predictor affects the target).
- Builds robust models with better generalization.

7.2 What is the penalty?

In normal linear regression the goal is:

```
Minimize SSE = sum (Actual - Predicted)^2
```

In ridge regression, a penalty is added:

```
Minimize: SSE + lambda * sum (b1^2 + b2^2 + ... + bn^2)
```

- SSE = sum of squared errors (how wrong the model is)
- b1, b2, …, bn = model coefficients (weights for each variable)
- lambda = penalty strength

7.3 Why does shrinking help?

Problem with large coefficients:

- The model reacts too strongly to small changes in the data.
- That leads to **overfitting**: the model fits the noise, not just the pattern.

Solution with ridge:

- Ridge shrinks the b values.
- Smaller b means less sensitivity to noise, so a simpler, more stable model.

7.4 Lambda: the tuning key

What happens if:

- lambda = 0: no penalty, same as normal regression
- lambda small (e.g., 0.01): very light shrinking
- lambda large (e.g., 100): very strong shrinking (coefficients close to 0)

Lambda	Effect	Coefficients
0	No penalty, normal regression	Original b
Small	Light penalty	Slightly shrunk b
Medium	Balanced penalty (good performance)	Well-shrunk b
Large	Strong penalty	All b close to 0

Intuition: lambda is a "leash" on the coefficients. No leash (0) = coefficients run wild. Very short leash = they cannot move from 0. We want a **medium leash**, and cross-validation finds it for us.

7.5 Code in R

7.5.1 Read the data

```
data1 <- read.csv("../data/raw/agricultural_yield_data.csv")
head(data1)
```

```
  Soil_Nitrogen Soil_Phosphorus Rainfall Temperature Yield
1         89.86           47.31   725.83       23.62 8.945
2         93.35           49.13   681.58       22.85 8.641
3         81.20           39.59   835.36       18.65 7.964
4        111.77           57.23   856.67       23.70 9.757
5         74.06           33.35   828.79       23.45 6.853
6         60.71           28.23   696.58       22.10 6.482
```

First check that multicollinearity is really present here:

```
lm_check <- lm(Yield ~ Soil_Nitrogen + Soil_Phosphorus + Rainfall + Temperature, data = data1
round(vif(lm_check), 2)
```

```
  Soil_Nitrogen Soil_Phosphorus        Rainfall     Temperature
           5.52            5.48            1.59            1.09
```

7.5.2 Prepare predictor matrix and response vector

```
a <- as.matrix(data1[, c("Soil_Nitrogen", "Soil_Phosphorus", "Rainfall", "Temperature")])
b <- data1$Yield
```

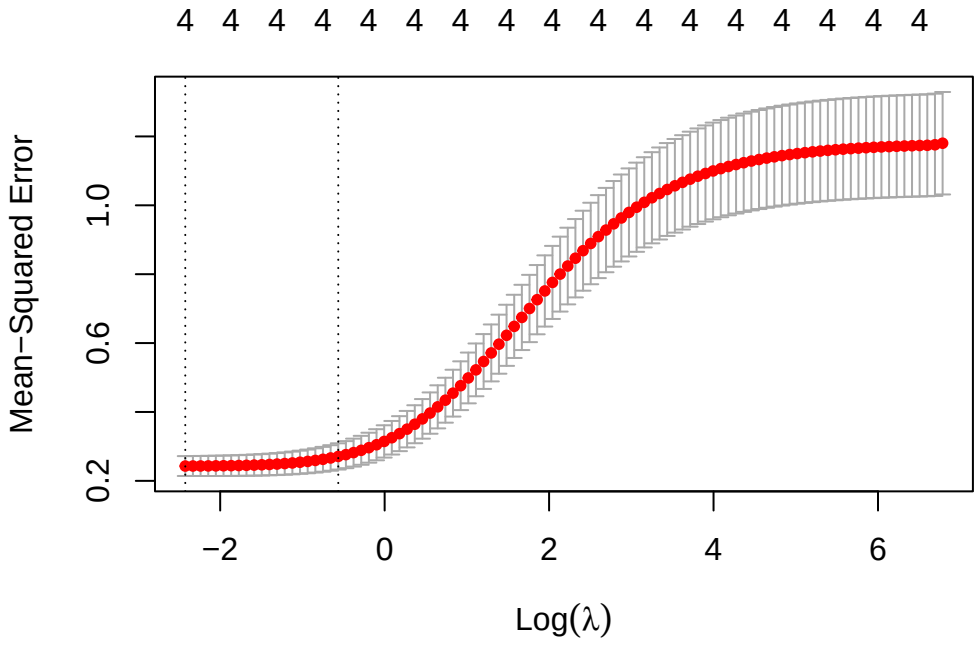
- `a` extracts the predictor variables and converts them into a **matrix** (glmnet requires predictors in matrix form).
- `b` is the response vector (yield).

7.5.3 Fit ridge regression with cross-validation to find the best lambda

```
set.seed(123)
ridge_model <- cv.glmnet(a, b, alpha = 0)
```

- Fits the ridge regression model using **cross-validation** (`cv.glmnet`).
- `alpha = 0` tells glmnet to perform **ridge** (if `alpha = 1`, it is lasso).
- Cross-validation automatically splits the data, fits models with different lambda values (penalties), and selects the **best lambda** that minimizes prediction error.

```
plot(ridge_model)
```



How to read this plot: the x-axis is log(lambda) and the y-axis is prediction error. The left dotted line is `lambda.min` (lowest error). The error curve is lowest at a **medium** lambda, high at tiny lambda (overfit) and high at huge lambda (over-shrunk).

7.5.4 Extract the best lambda value

```
best_lambda <- ridge_model$lambda.min
best_lambda
```

```
[1] 0.08853264
```

7.5.5 Predict yield using the ridge model and best lambda

```
Predicted_yield <- predict(ridge_model, s = best_lambda, newx = a)
head(Predicted_yield)
```

```
          s1
[1,] 8.551899
[2,] 8.544574
[3,] 7.932267
[4,] 9.838981
[5,] 7.804122
[6,] 6.836030
```

7.6 Seeing the shrinkage

Compare ordinary regression coefficients with ridge coefficients:

```
lm_coef    <- coef(lm_check)
ridge_coef <- as.matrix(coef(ridge_model, s = best_lambda))

shrink_tbl <- data.frame(
  Term = rownames(ridge_coef),
  OLS = round(as.numeric(lm_coef[rownames(ridge_coef)]), 4),
  Ridge = round(as.numeric(ridge_coef), 4)
)
kable(shrink_tbl)
```

Term	OLS	Ridge
(Intercept)	0.5296	0.8266

Term	OLS	Ridge
Soil_Nitrogen	0.0208	0.0216
Soil_Phosphorus	0.0521	0.0474
Rainfall	0.0027	0.0026
Temperature	0.0758	0.0705

Read it like this: ridge coefficients are **pulled toward 0** compared with ordinary regression, especially for the overlapping predictors (the pair with high VIF). No variable is removed.

And the effect of lambda itself (light, balanced, strong):

```
lam <- c(light = min(ridge_model$lambda),
         balanced = best_lambda,
         strong = max(ridge_model$lambda))

lam_tbl <- sapply(lam, function(l) as.numeric(coef(ridge_model$glmnet.fit, s = l)))
rownames(lam_tbl) <- rownames(ridge_coef)
kable(round(lam_tbl, 4))
```

	light	balanced	strong
(Intercept)	0.8266	0.8266	7.9864
Soil_Nitrogen	0.0216	0.0216	0.0000
Soil_Phosphorus	0.0474	0.0474	0.0000
Rainfall	0.0026	0.0026	0.0000
Temperature	0.0705	0.0705	0.0000

As lambda goes from light to strong, all coefficients shrink toward 0. This is the lambda table from the theory, made real.

7.7 How well does it predict?

```
actual <- data1$Yield
rmse_ridge <- sqrt(mean((actual - as.numeric(Predicted_yield))^2))
rmse_lm    <- sqrt(mean(residuals(lm_check)^2))
kable(data.frame(Model = c("Ordinary regression", "Ridge"),
                 RMSE = round(c(rmse_lm, rmse_ridge), 4)))
```

Model	RMSE
Ordinary regression	0.4588
Ridge	0.4601

Ridge usually has **slightly higher error on the training data** (it is not trying to fit every detail). Its advantage is **stable coefficients and better performance on new data**.

8 Which Fix to Use?

Fix	Best when	Cost
More data	You can still collect it	Time and money
Remove one variable	One variable is clearly redundant	Lose information on that variable
Combine variables	Variables measure the same idea and are on the same scale	Hard to interpret
PCA	Many overlapping predictors	New variables are hard to explain
Ridge	You want to keep all variables and need stable coefficients	Coefficients are shrunk (slightly biased)

9 Key Takeaways

1. High multicollinearity can be managed by more data, removing or combining variables, PCA, or ridge.
2. **PCA:** standardize with `scale()`, run `prcomp()`, check the proportion of variance (more than 70%, aim for 80% or more), then use `pca_result$x[, 1]` in `lm()`.
3. **Ridge:** adds a penalty `lambda * sum(b^2)` that shrinks coefficients; it keeps all variables.
4. **Lambda:** 0 = normal regression, small = light shrink, medium = balanced, large = coefficients near 0.
5. `cv.glmnet(a, b, alpha = 0)` finds the best lambda by cross-validation (`lambda.min`); `alpha = 1` would be lasso.